

Machine Learning

* NumPy, Matplotlib, Pandas & other necessary libraries.

NumPy `import numpy as np`

→ Creates an array of one with desired shape

`ones = np.ones(shape, dtype=int);`

* `np.hstack((array-we-want-to-stack, target-array));`

↳ `hstack` takes a tuple of arrays as arguments

alternatives:

- ① `np.c_[ones, original-array];` → Does column wise concatenation
- ② `np.insert(original-array, index, value, axis=1);` → More control.

Pandas `import pandas as pd;`

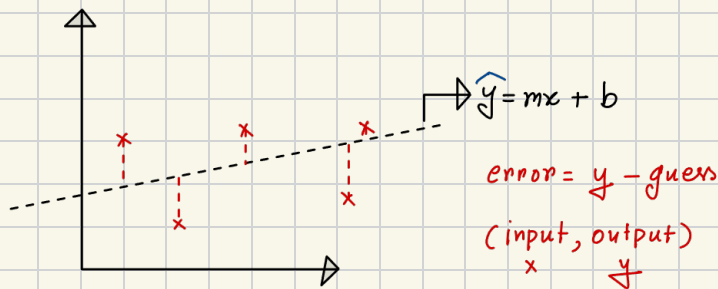
`df = pd.read_csv("file_name", header = None, sep = ",");`

Loads without headers

`columns = ['a', 'b', 'c', 'd', 'e', 'f']`

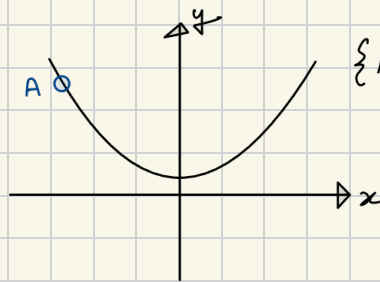
`df.columns = columns` → Inserts columns

Linear Regression with GD



Cost: $\sum_{i=0}^n (\hat{y}_i - y_i)^2$ We want to minimise this function.

$\therefore \text{Cost} = \sum (\hat{y}_i - y_i)^2$ this is kind of like $y = x^2$



{ Now, we need to find the value of x for that y is lowest.

Suppose, we're at point A & if we want to reduce the function, we need to find its gradient. But how do we go down?

→ We calculate the slope.

$$m = m + \Delta m$$

$$b = b + \Delta b$$

$$\therefore J(m, b) = \frac{1}{n} \sum_{i=0}^n (mx + b - y)^2$$

$$\frac{\partial}{\partial m} J = 2(mx + b - y)x \quad \& \quad \frac{\partial}{\partial b} J = 2(mx + b - y)(1)$$

* $\frac{1}{n}$ in $J(m,b) = \frac{1}{n} \sum (\hat{y}_i - y_i)^2$?

→ We want to take the average error & minimise it.

→ Makes it scale irrelevant so that same learning rate works in different datasets.

We also sometimes take $\frac{2}{n}$, this is purely because of algebraic convenience.

$$\text{Update rule: } m^{(t+1)} = m - \alpha \frac{\partial}{\partial m} J$$

$$b^{(t+1)} = b - \alpha \frac{\partial}{\partial b} J$$

* Example:

$(x,y) = \{(1,2), (2,3), (3,5)\}$ predict y when $x=7$

here, $n=3$, model, $\hat{y} = mx + b$ let, $\hat{y}_1 = \hat{y}_2 = \hat{y}_3 = 0$ { because, we assumed $m=0$ & $b=0$ }

$$J = \frac{1}{n} \sum_0^n (\hat{y}_i - y_i)^2 = \{(0-2)^2 + (0-3)^2 + (0-5)^2\} / 3$$

$$= \frac{4+9+25}{3} = \frac{38}{3} \approx 12.67$$

let, $\alpha = 0.01$

Now, we'll minimise loss by updating m & b

$$\text{Given, } J = \frac{1}{n} \sum_0^n (mx + b - y_i)^2$$

$$\therefore \frac{\partial J}{\partial m} = \frac{1}{n} \sum (mx_i + b - y_i) \times 2 \times (x_i)$$

$$\& \frac{\partial J}{\partial x} = \frac{2}{n} \sum (mx_i + b - y_i) (1)$$

$$m^{(t+1)} = m^t + \alpha \frac{\partial}{\partial m} J$$

$$b^{(t+1)} = b^t + \alpha \frac{\partial}{\partial b} J$$

1st step,

$$\frac{\partial J}{\partial m} = \frac{1}{3} \{ (0 \times 1 + 0 - 2) \times 2(1) + (0 \times 2 + 0 - 3)(2 \times 2) + (-5)(2 \times 3) \} = -15.33$$

$$\text{Made with Goodnotes} \quad m^1 = m^0 - \alpha \left(\frac{\partial J}{\partial m} \right) = 0 - 0.01(-15.33) = 0.1533$$

let's get b^1

$$\frac{\partial J}{\partial b} = \frac{2}{n} \sum_0^n (mx_i + b - y_i) = \frac{2}{3} (-2 - 3 - 5) = -6.667$$

$$\therefore b^1 = b^0 - \alpha \left(\frac{\partial J}{\partial b} \right) = 0 - 0.0667 = 0.0667$$

So, model updates to,

$$\hat{y}_i = 0.1533 x_i + 0.0667$$

$$\therefore \hat{y}_1 = 0.22 \quad \hat{y}_2 = 0.3733 \quad \hat{y}_3 = 0.5266$$

$$\begin{aligned} \text{Current error: } J &= \frac{1}{n} \sum_0^n (\hat{y}_i - y_i)^2 \\ &= \frac{1}{3} \left((0.22 - 2)^2 + (0.3733 - 3)^2 + (0.5266 - 5)^2 \right) \\ &= \frac{\{(-1.78)^2 + (-2.6267)^2 + (-4.4734)^2\}}{3} \\ &= 10.022067 \end{aligned}$$

So, we've reduced error in step 1 but we have two more steps to go. It took a lot of manual calculations, let's simplify a bit using matrices.

* Dataset $\{(1,2), (2,3), (3,5)\}$ Δ prediction $\hat{y}_i = mx_i + b$

$$\theta = \begin{pmatrix} m \\ \theta \end{pmatrix} \rightarrow \overline{y} = \overline{x} \overline{\theta}$$

* Design matrix:

$$\begin{aligned} \overline{x} &= \begin{pmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{pmatrix} \quad \overline{\theta} = \begin{pmatrix} m \\ \theta \end{pmatrix} \quad \overline{y} = \overline{x} \overline{\theta} = \begin{pmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{pmatrix} \begin{pmatrix} m \\ \theta \end{pmatrix} \\ &= \begin{pmatrix} m + \theta \\ 2m + \theta \\ 3m + \theta \end{pmatrix} \end{aligned}$$

as $m, \theta = 0, 0$

$n = 3$

$$\therefore \bar{y} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \quad \text{Given, } \bar{y} = \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix}$$

$$\begin{aligned} \bar{e} &= \bar{y} - y = \begin{pmatrix} -2 \\ -3 \\ -5 \end{pmatrix} & \bar{e}^2 &= (\bar{y} - y)^2 = \|\bar{y} - y\|_2 \\ & & &= (\bar{y} - y)^T (\bar{y} - y) \\ & & &= (2 \ 3 \ 5) \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix} \\ & & &= (4 + 9 + 25) = 38 \end{aligned}$$

* Cost Function,

$$J = \frac{\bar{e}^2}{n} = 38/3 = 12.667$$

* Gradient,

$$\begin{aligned} J(\theta) &= 1/n (\bar{y} - y)^T (\bar{y} - y) = \frac{1}{n} (\bar{x}\theta - \bar{y})^T (\bar{x}\theta - \bar{y}) \\ &= \frac{1}{n} (\bar{x}^T \theta^T - \bar{y}^T) (\bar{x}\theta - \bar{y}) \\ &= \frac{1}{n} (\theta^T \bar{x}^T \bar{x} \theta - \bar{x}^T \theta^T \bar{y} - \bar{y}^T \bar{x} \theta - \bar{y}^T \bar{y}) \\ &= \frac{1}{n} (\theta^T \bar{x}^T \bar{x} \theta - 2 \bar{x}^T \theta^T \bar{y} - \bar{y}^T \bar{y}) \end{aligned}$$

$J(\theta) = \frac{1}{n} (\theta^T \bar{x}^T \bar{x} \theta - 2 \bar{y}^T \bar{x} \theta - \bar{y}^T \bar{y})$

* Matrix Calculus Identities

$$\begin{aligned} \textcircled{1} \quad \nabla_{\theta} (\theta^T A \theta) &= 2A\theta \\ \textcircled{2} \quad \nabla_{\theta} (c^T \theta) &= c \\ \textcircled{3} \quad \nabla_{\theta} (\text{constant}) &= 0 \end{aligned}$$

{ When A is symmetric & if $A = x^T x$ then A is always symmetric }

→ Appendix 1

* Applying these in the cost function,

$$\textcircled{1} \quad \nabla_{\theta} (\theta^T \bar{x}^T \bar{x} \theta) = 2 \bar{x}^T \bar{x} \theta$$

$$\textcircled{2} \quad \nabla_{\theta} (2 \bar{y}^T \bar{x} \theta) = 2 \bar{x}^T \bar{y}$$

$$\textcircled{3} \quad \nabla_{\theta} (\bar{y}^T \bar{y}) = 0$$

$$\begin{aligned} \therefore \nabla_{\theta} J &= (2 \bar{x}^T \bar{x} \theta - 2 \bar{x}^T \bar{y}) / n \\ &\downarrow \text{Gradient (w.r.t } n \& b) \\ &= \frac{2 \bar{x}^T}{n} (\underbrace{\bar{x} \theta - \bar{y}}_{\text{Error Vector}}) \end{aligned}$$

Update rule: $\theta := \theta - \alpha \nabla_{\theta} J$

 Let's verify!

$$\{(1, 2), (2, 3), (3, 5)\}$$

$$\begin{array}{l} \underset{A}{\bar{x}} = \begin{pmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{pmatrix} \quad \underset{B}{\bar{\theta}} = \begin{pmatrix} m \\ b \end{pmatrix} \quad \underset{C}{\bar{y}} = \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix} \quad \hat{y} = \bar{x} \bar{\theta} \quad \bar{e} = \hat{y} - y \\ \quad \quad \quad = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \quad y_1 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \end{array}$$

$$\bar{e} = \begin{pmatrix} -2 \\ -3 \\ -5 \end{pmatrix}$$

* Const Calc

$$\begin{aligned} J(\theta) &= \frac{1}{n} (\bar{x}\theta - y)^T (\bar{x}\theta - y) \\ &= \frac{1}{3} \begin{pmatrix} -2 & -3 & -5 \end{pmatrix} \begin{pmatrix} -2 \\ -3 \\ -5 \end{pmatrix} = \frac{1}{3} (2^2 + 3^2 + 5^2) = 38/3 = 12.67 \end{aligned}$$

$$\begin{aligned} * \nabla_{\theta} J(\theta) &= \frac{2}{n} \bar{x}^T (\bar{x}\theta - y) \\ &= \frac{2}{3} \begin{pmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \end{pmatrix} \left(\begin{pmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \end{pmatrix} - \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix} \right) \\ &= \frac{2}{3} \begin{pmatrix} -23 \\ -10 \end{pmatrix} = \begin{pmatrix} -46/3 \\ -20/3 \end{pmatrix} = \begin{pmatrix} -15.33 \\ -6.667 \end{pmatrix} \end{aligned}$$

$$\therefore \nabla_{\theta} J(\theta) = \begin{pmatrix} -15.33 \\ -6.667 \end{pmatrix}$$

$$\begin{aligned} \therefore \theta^1 &= \theta - \alpha \begin{pmatrix} -15.33 \\ -6.667 \end{pmatrix} \quad \{ \alpha = 0.01 \} \\ &= \begin{pmatrix} 0.1533 \\ 0.6667 \end{pmatrix} \end{aligned}$$

$$\hat{y}_1 = \bar{x} \bar{\theta} \\ = \begin{pmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{pmatrix} \begin{pmatrix} 0.1533 \\ 0.0667 \end{pmatrix} = \begin{pmatrix} 0.22 \\ 0.3733 \\ 0.5266 \end{pmatrix}$$

$$e^1 = \begin{pmatrix} -1.78 \\ -2.626 \\ -4.9739 \end{pmatrix}$$

$$\begin{aligned} \theta^{(2)} &= \theta^{(1)} - \alpha \left(\nabla_{\theta} J(\theta) \right) = \theta^{(1)} - \alpha \left(\frac{2}{n} x^T (x \theta^{(1)} - y) \right) \\ &= \theta^{(1)} - 0.01 \left(\frac{2}{3} \begin{pmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \end{pmatrix} \left(\begin{pmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{pmatrix} \begin{pmatrix} 0.1533 \\ 0.0667 \end{pmatrix} - \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix} \right) \right) \\ &= \theta^{(1)} - 0.01 \left(\frac{2}{3} \begin{pmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} -1.78 \\ -2.626 \\ -4.9739 \end{pmatrix} \right) e^1 \end{aligned}$$

$$\therefore \theta^{(2)} = \begin{pmatrix} 0.289698 \\ 0.125896 \end{pmatrix}$$

$$\begin{aligned} \theta^3 &= \theta^{(2)} - 0.01 \times \frac{2}{3} \begin{pmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} -1.584 \\ -2.271 \\ -9.005 \end{pmatrix} \\ &= \begin{pmatrix} 0.289698 \\ 0.125896 \end{pmatrix} - \begin{pmatrix} -0.121263 \\ -0.052563 \end{pmatrix} = \begin{pmatrix} 0.410912 \\ 0.1734 \end{pmatrix} \end{aligned}$$

$$e = x\theta - y$$

$$\begin{aligned} J(\theta)^{(3)} &= \left| \bar{x} \theta^{(3)} - y \right|_2 \times \frac{1}{n} \quad (n=3) \\ &= 6.29 \end{aligned}$$

**How long will we go?*

→ Our error term is reducing, so we can go as low as we want till error ≈ 0

But, we can assign few other stopping conditions

- ① if $J(\theta) \approx 0$
- ② iteration count limit
- ③ $\|e\|_2 \approx 0$
- ④ $\|J(\theta)^{k+1} - J(\theta)^{(k)}\| < \epsilon$ or ≈ 0

But no matter what we do, we need computational power because datasets can become very large!

→ We'll use python to create our linear regress with GD algorithm, calculate accuracy.

* Appendix 1

$$\textcircled{1} \quad \theta = \begin{pmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_d \end{pmatrix} \in \mathbb{R}^{d \times 1} \quad \nabla_{\theta} f(\theta) = \begin{pmatrix} \partial f / \partial \theta_1 \\ \partial f / \partial \theta_2 \\ \vdots \\ \partial f / \partial \theta_d \end{pmatrix}$$

$$* \frac{d}{d(\text{vector})}(\text{vector}) = (\text{vector})$$

$$\textcircled{2} \quad \nabla_{\theta}(c^T \theta) = c$$

$$c^T \theta = \sum_{i=0}^d c_i \theta_i \quad \frac{\partial}{\partial \theta_j}(c^T \theta) = c_j$$

Stacking all in c

$$\textcircled{3} \quad \nabla_{\theta}(k) = 0$$

Constant rule

$$\textcircled{4} \quad \nabla_{\theta} \theta^T A \theta = 2A\theta \text{ if } A \text{ is symmetric.}$$

Linear Regression = Quadratic Optimisation

Convex = Single Global Minima

Normal Equation (for any number of features)

$$\theta = (X^T X)^{-1} X^T y$$

$$m = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

Stochastic Gradient Descent

→ Regular GD uses all datapoints to calculate the gradient but this becomes costly over time as n increases.

→ SGD uses one datapoint at a time.

*Steps:

① Define Per-Sample Loss:

$$l_i(\theta) = (x_i^T \theta - y_i)^2$$

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n l_i(\theta)$$

② Gradient of One Sample (x_i, y_i)

$$\nabla_{\theta} l_i(\theta) = 2(x_i^T \theta - y_i)(x_i)$$

③ Update rule

$$\theta \leftarrow \theta - \alpha \cdot \underbrace{2x_i(x_i^T \theta - y_i)}_{\nabla_{\theta} l_i(\theta)}$$

Example:

$$\hat{y} = mx + b \quad x_i = [x_i, 1] \quad \theta = \begin{bmatrix} m \\ b \end{bmatrix}$$

i	x	y
1	1	2
2	2	3
3	3	5

$$\theta^{(0)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

$$\alpha = 0.1$$

One random point $(x_2, y_2) = (2, 3)$

Design Matrix $x_2 = \begin{pmatrix} 2 & 1 \end{pmatrix}$

$$\hat{y}_2 = x_2 \theta = \begin{pmatrix} 2 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \end{pmatrix} = 0 + 0 = 0$$

$$e_2 = 0 - 3 = -3 = x_2 \theta - y_2$$

$$G = \text{Gradient} = \nabla_{\theta} l_2(\theta) = 2x_2^T e_2$$

$$= 2 \begin{pmatrix} 2 \\ 1 \end{pmatrix} (-3) = \begin{pmatrix} -12 \\ -6 \end{pmatrix}$$

$$\text{Update: } \theta^{(1)} = \theta - \alpha G = \begin{pmatrix} 0 \\ 0 \end{pmatrix} - (0.1) \begin{pmatrix} -12 \\ -6 \end{pmatrix} = \begin{pmatrix} 1.2 \\ 0.6 \end{pmatrix}$$

* 2nd Iteration $\{ \text{Random point } (x_1, y_1) = (1, 1) \text{ Design Matrix } \begin{pmatrix} 1 & 1 \end{pmatrix} \}$

$$x_1 = \begin{pmatrix} 1 & 1 \end{pmatrix} \quad y_1 = 2$$

$$y_1 = \begin{pmatrix} 1 & 1 \end{pmatrix} \begin{pmatrix} 1.2 \\ 0.6 \end{pmatrix} = 1.2 + 0.6 = 1.8$$

$$e = 1.8 - 2 = -0.2$$

$$G = 2x_1^T e = 2 \begin{pmatrix} 1 \\ 1 \end{pmatrix} (-0.2) = \begin{pmatrix} -0.4 \\ -0.4 \end{pmatrix}$$

$$\theta^{(2)} = \theta^{(1)} - \alpha G$$

$$= \begin{pmatrix} 1.2 \\ 0.6 \end{pmatrix} - 0.1 \begin{pmatrix} -0.4 \\ -0.4 \end{pmatrix} = \begin{pmatrix} 1.24 \\ 0.64 \end{pmatrix}$$

* Although S-GD never truly settles

$$\alpha_t = \frac{\alpha_0}{1+kt} \quad \text{so we need to sometimes implement}$$

learning rate decay.

Takeaways

- ① SGD uses one sample.
- ② Each update is cheap but noisy.
- ③ Over many updates noise averages out.
- ④ Learning rate must be smaller than VGD.

Mini Batch Gradient Descent

→ Similar concept as VGD but this time we work with a sub-sample.

Algorithm: ① Select sample size.

② Select sub-dataset randomly.

③ Find gradient of that sub-dataset.

④ Shuffle dataset.

⑤ Found desired expected loss? (JO). Yes! stop.

No! → Goto Step 2.

Adaptive Gradient Descent AdaGrad

→ Look we're basically calculating how fast objects move in which direction. We are talking about shape of the loss function. Hessian simply describe that shape.

→ Gradient tells about the direction of the steepest descent, but tells nothing about 1) how curved the surface is.
2) whether the direction is safe to move.

Two very different surfaces can have the same gradient at a point.

Appendix 2

Suppose a 1D function $f(x) = ax^2$ $a=100 \rightarrow$ very steep.
 $a=0.01 \rightarrow$ very flat.

$f'(x) = 2ax \rightarrow$ This is the gradient

But what determines how fast the gradient changes?

$f''(x) = 2a \rightarrow$ This!

Hessians are basically second derivatives arranged into a matrix.

In gradient descent

$$\theta_{t+1} = \theta_t - \alpha \nabla J(\theta_t)$$

AdaGrad basically gives each parameter its own learning rate based on historical gradients.

① $g_t = \nabla_{\theta} J(\theta_t) \in \mathbb{R}^d \rightarrow$ Gradient of loss function

② Accumulating squared gradients

$$G_t = \sum_{i=1}^t g_i \odot g_i = \bar{g}_t^T \cdot \bar{g}_t$$

$\odot \rightarrow$ Piecewise multiplication
 $G_t \in \mathbb{R}^d$

③ Update rule:

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{G_t} + \epsilon} \quad ; \quad \text{Division is element-wise.}$$

ϵ is a small constant (so we don't get ∞ error)

Why squared gradients?

i) Makes all contributions positive.

ii) Penalises large gradients disproportionately.

Because

Large gradients $\rightarrow$ Dominate the sum $\rightarrow$ shrink step size fast
vice-versa.

$H \approx \text{diag}(\sum g^2) \rightarrow$ AdaGrad estimates Hessians basically because
calculating Hessians are 1) Expensive
2) Hard 3) Unstable

Weakness

if $G_t \uparrow$ monotonically then $\frac{1}{\sqrt{G_t}} \downarrow$ & eventually learning
rate becomes 0.

Example: $(1,2), (2,3), (3,5)$. Simulate 1R with AdaGrad.

$\rightarrow$ model: $y = \theta_1 x + \theta_0$ $\theta = \begin{pmatrix} \theta_0 \\ \theta_1 \end{pmatrix}$

Design matrix: $\begin{pmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{pmatrix} x$

$$J(\theta) = \frac{1}{2m} \sum_{i=0}^m (\hat{y}^{(i)} - y^{(i)})^2$$

$$= \frac{1}{2m} (\hat{y} - y)^T (\hat{y} - y)$$

$$= \frac{1}{2m} (x\theta - y)^T (x\theta - y)$$

$$= \frac{1}{2m} (x^T \theta^T - y^T) (x\theta - y)$$

$$= \frac{1}{2m} (\theta^T x^T x \theta - 2 y^T x \theta + y^T y)$$

$$\theta = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

$$\hat{y} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

$$= x\theta$$

$$e = \hat{y} - y$$

$$\nabla_{\theta} J(\theta) = \frac{1}{2m} (2 x^T x \theta - 2 x^T y) = \frac{1}{m} (x^T x \theta - x^T y)$$

$$G_t = g_k^T \cdot g_k$$

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{G_t} + \epsilon}$$

RMSProp

Why AdaGrad fails?

$$\textcircled{1} G_t = \sum_{k=1}^t g_k \odot g_k \quad \& \quad \theta_t = \theta_t - \eta \frac{g_t}{\sqrt{G_t + \epsilon}}$$

G_t increases monotonically

We don't need information about all past gradients rather we need information only about recent curvature.

$s_t \rightarrow$ Running estimate of curvature magnitude per parameter

$s_{t,j} = \mathbb{E}[g_{t,j}^2]$ "How large are the gradients for parameter θ_j , on average, recently?"

$$\text{RMS}(g_j) = \sqrt{s_{t,j}}$$

$$g_t = \nabla_{\theta} J(\theta_t) = \frac{1}{m} x^T (x \theta_t - y)$$

$$s_t = \rho s_{t-1} + (1-\rho) g_t \odot g_t \quad \rho \in (0, 1)$$

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{s_t + \epsilon}} \quad s_t \in \mathbb{R}$$

$$J(\theta) \quad \rightarrow \quad \theta_{t+1} = \theta_t - \eta (\text{diag}(\sqrt{s_t})^{-1}) g_t$$

* RMSProp performs adaptive diagonal second order scaling.

* Numerical Example

$(1, 2), (2, 3), (3, 5)$

$$x = \begin{pmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{pmatrix} \quad y = \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix}$$

* initialisation

$$\eta = 0.1 \quad s_0 = 0$$

$$\rho = 0.9$$

$$\epsilon \approx 0$$

$$J(\theta) = \frac{1}{n} (x\theta - y)^T (x\theta - y)$$

$$= \frac{1}{n} (\theta^T x^T x \theta - x^T \theta^T y - x \theta y^T - y^T y)$$

$$g_t = \nabla_{\theta} J(\theta) = (2 x^T x \theta - 2 x^T y - 0) / n$$

$$= \frac{2}{n} x^T \underbrace{(x\theta - y)}_{\text{Error}}$$

↙ $n=3$

$$r_t = \rho \cdot r_{t-1} + (1-\rho) g_t \odot g_t$$

$$\theta_t = \theta_{t-1} - \eta \frac{g_t}{\sqrt{r_t} + \epsilon}$$

$$g_t = \begin{pmatrix} -15.33 \\ -6.667 \end{pmatrix}$$

$$r_t = \rho \cdot r_{t-1} + (1-\rho) g_t \odot g_t$$

$$= 0.9 \times \begin{pmatrix} 0 \\ 0 \end{pmatrix} + (0.1) \begin{pmatrix} (-15.33)^2 \\ (-6.667)^2 \end{pmatrix}$$

$$= \begin{pmatrix} 235.111 \\ 44.449 \end{pmatrix}$$

$$\sqrt{r_t} = \begin{pmatrix} 4.897 \\ 2.108 \end{pmatrix} \quad \frac{g_t}{r_t} = \begin{pmatrix} -15.33 / 4.897 \\ -6.667 / 2.108 \end{pmatrix} = \begin{pmatrix} -3.164 \\ -3.163 \end{pmatrix}$$

$$\theta = \begin{pmatrix} 0 \\ 0 \end{pmatrix} - 0.01 \begin{pmatrix} -3.164 \\ -3.163 \end{pmatrix}$$

Momentum Descent

{ heavy ball method }

Plain gradient descent: $\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$

But has ① ill conditioned Hessians.

② Oscillate across steep directions.

③ Move slowly along flat directions.

Momentum fixes this by accumulating velocity, not just reaching to the current gradient.

Analogy: ① Gradient $\rightarrow$ Force

② Momentum $\rightarrow$ Velocity

③ Parameters $\rightarrow$ Position

$$V^* \in \mathbb{R}^d$$

$\mu \rightarrow$ Momentum

(β) Coefficient (0.9)

$$F = m \frac{dv}{dt} \Rightarrow F dt = m dv$$

Momentum is exponentially weighted moving average of past gradients.

* Algebraic form

θ_0 given, $v_0 = 0$

* Update Rule

$$v_t = \mu v_{t-1} + g_t = \mu (\mu v_{t-2} + g_{t-1}) + g_t = \dots \sum_{k=0}^t \mu^k g_{t-k}$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

$$v_t = \beta v_{t-1} + (1-\beta) \nabla_{\theta} J(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

* Example: $\{(1,2), (2,3), (3,5)\} \rightarrow$ Fitting linear regression with momentum GD.

$\rightarrow$ model: $y = wx + b$ $\theta = \begin{pmatrix} w \\ b \end{pmatrix}$

$$\begin{aligned} \mathcal{L} &= J(\theta) = \frac{1}{2m} \sum_0^m (\hat{y}_i - y)^2 \\ &= \frac{1}{2m} (\hat{y} - y)^T (\hat{y} - y) \\ &= \frac{1}{2m} (x\theta - y)^T (x\theta - y) \\ &= \frac{1}{2m} (\theta^T x^T x \theta - x^T \theta^T y - y^T x \theta - y^T y) \\ &= \frac{1}{2m} (\theta^T x^T x \theta - 2x^T \theta^T y - y^T y) \end{aligned}$$

initialisations

$$\theta = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \quad v = \begin{pmatrix} v_w \\ v_b \end{pmatrix} \in \mathbb{R}^2$$

$$\eta = 0.1$$

$$\beta = 0.9$$

$$x = \begin{pmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{pmatrix} \in \mathbb{R}^{3 \times 2}$$

$$y = \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix}$$

$$\hat{y} = x\theta$$

$$\nabla_{\theta} J(\theta) = \frac{1}{m} x^T (x\theta - y)$$

* Update rules:

$$v_t = \beta v_{t-1} + (1-\beta) \nabla J(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

* Predictions: $x\theta = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} = \hat{y}_0$ $e_0 = \begin{pmatrix} -2 \\ -3 \\ -5 \end{pmatrix}$

1st iteration

$$\nabla J(\theta) = \frac{1}{3} \begin{pmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} -2 \\ -3 \\ -5 \end{pmatrix} = \begin{pmatrix} -7.667 \\ -0.333 \end{pmatrix}$$

$$v_1 = (0.9) \begin{pmatrix} 0 \\ 0 \end{pmatrix} - (1-0.9) \begin{pmatrix} -7.667 \\ -0.333 \end{pmatrix} = \begin{pmatrix} -0.767 \\ -0.333 \end{pmatrix}$$

$$\theta_1 = \begin{pmatrix} 0 \\ 0 \end{pmatrix} - 0.1 \begin{pmatrix} 0.767 \\ 0.333 \end{pmatrix} = \begin{pmatrix} 0.0767 \\ 0.0333 \end{pmatrix}$$

Nesterov Momentum

- Lookahead Momentum
- Rather than pressing break while you're on top of the slope, press break when you see the slope.

In classical momentum descent

$$v_t = \beta v_{t-1} + (1-\beta) \nabla J(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

The gradient is evaluated at current point (θ_t) & this can

① Overshoot.

② Oscillate near the minimum.

But in Nesterov

$$\theta_{\text{ea}} = \theta - \beta \cdot \eta \cdot v$$

$$\frac{1}{2m} (x^T \cdot (x \cdot \theta_{\text{ea}} - y))$$

$$g_t = \nabla J(\theta_t - \eta \beta v_{t-1}) \quad \# \text{ The point we look ahead.}$$

$$v_t = \beta v_{t-1} + (1-\beta) g_t$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

Actual θ is calculated here!

} Rest stays same

ADAM

→ Adaptive Moment Estimation

↳ Momentum Descent + RMSProp

* Parameters

$$\theta^* \in \mathbb{R}^n$$

$$g^* = \nabla_{\theta} J(\theta^*)$$

* Hyperparameters

$$\alpha = 0.001$$

$$\beta_1 = 0.9 \text{ Momentum}$$

$$\beta_2 = 0.999 \text{ RMS Decay}$$

$$\epsilon = 10^{-8}$$

* Initialisations

$$\theta_0 = 0, m_0 = 0, v_0 = 0$$

* Steps

$$1 \quad m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad \{\text{Momentum}\}$$

$$2 \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t \odot g_t \quad \{\text{Variance}\}$$

3 Bias Correction

$$\hat{m}_t = \frac{m_t}{1 - \beta_1}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2}$$

4 Parameter Update

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$